

拼团 / 团购

社交裂变 vs 商家库存利用率——gomall v2 业务侧

gomall · 业务侧 deck

今日大纲

业务困局：为什么拼团是一个独立业务线

业务流程：从发起到散团的 4 个节点

库存博弈：reserved 桶在团购中的关键作用

角色 1：C 端裂变与商家库存利用率

角色 2：客服 SOP 与业务码对照

角色 3：黑产场景与 gomall 怎么挡

角色 4：SRE 视角——Saga 散团的可观测性

代码全景与测试

Q&A

业务困局：为什么拼团是一个独立业务线

拼团不是促销活动，是一种履约前置的销售模型

- 普通下单：用户决策 □ 单人付款 □ 商家发货。每一笔订单都是**孤立事件**。
- 拼团下单：用户决策 □ 多人凑齐 □ 一起发货。订单之间**绑成一组**，要么集体推进、要么集体退款。
- 业务关键：拼团把**用户社交关系**变成商家**库存利用率**的杠杆——商家用更低的价格换确定性销量。
- 5 角色看到 5 件不同的事，这就是为什么团购必须是独立 deck。

service/groupbuy.go::CreateGroup / JoinGroup / MarkGroupSuccess / ExpireGroup 四个入口

拼团的 5 角色视图

角色	拼团对他意味着什么	他最怕看到什么
C 端用户	价格更低 + 拉好友的社交奖励	凑不齐人 / 钱被锁 / 退款慢
商家	用折扣换 确定性 销量与提前备货	拼团价低 + 散团率高 = 库存空转
运营	GMV 杠杆——1 个团长拉来 N-1 个新客	黑产刷团把库存洗掉但不真付
客服	散团时 每用户都来问退款	没有标准 SOP 解释”为什么团没拼成”
SRE	散团是 Saga，必须库存归还 + 订单关闭	散团漏退 = 库存泄漏 + 用户投诉

核心矛

盾：低价的代价是**延后收单**（24h）+ **Saga 散团成本**。本 deck 每一帧都会回答：“这个设计满足了谁，牺牲了谁。” README 业务全景 5 角色表；本 deck 业务码 81001-81004 客服话术 <pkg/e/msg.go:58-61>

业务承诺：拼团接口的 SLO

接口	等级	可用率	p99 延迟
/groupbuy/:id (展示)	P1	99.9%	< 200ms
/groupbuy/create	P0 □ 1	99.95%	< 300ms
/groupbuy/:id/join	P0 □ 1	99.95%	< 100ms
散团 cron (24h ± 5min)	P1	99.9%	N/A

- join 是裂变流量的爆点（分享链接病毒式传播）——p99 必须低于 100ms，不然分享体验崩。
- 散团 cron 兜底 SLA **24h ± 5min** ——客服话术里给”1-3 工作日退款”，留出运营缓冲。
- 对照实测：/orders/create 50K RPS / p95 2.33ms，/groupbuy/join 因为多了 groupbuy_group 行 UPDATE，p95 预估 4-6ms，仍在 SLO 内。

业务边界：拼团这次做什么、不做什么

N 人成团 (固定)

24h 截单线

散团库存归还

分享落地页

多商品组合团

砍价 (变价)

团购返利

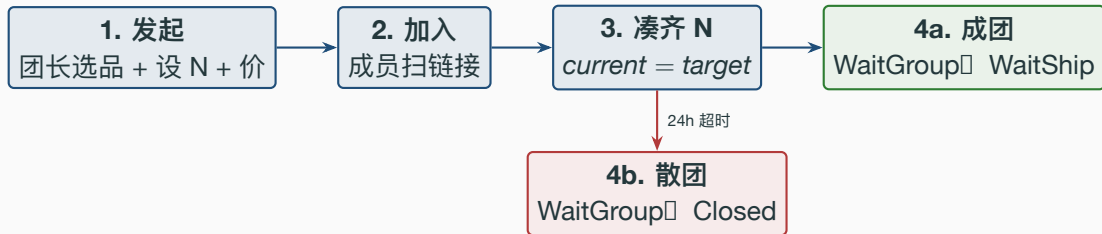
老带新奖励

- **做 (绿)**: 单 SKU 固定人数团 / 24h 散团 / 库存 Saga / 公开分享落地页。
- **不做 (红)**: 多 SKU 组合团 (业务复杂度爆炸)、变价砍价 (钱包系统未对齐)、返利与老带新奖励 (钱包未落地)。
- 业务诚实: 等钱包服务上线再迭代, 本期 MVP 聚焦”拼团核心闭环”。

`api/v1/groupbuy.go` 仅有 3 个 `handler`; `README` 业务边界一节列出”未做清单”

业务流程：从发起到散团的 4 个节点

业务侧主流程：1 个团长 + N-1 个团员的协同时序



- 4a 成团：所有成员订单进 WaitShip，库存 reserved $\rightarrow$ sold（真扣）。商家工作台开始看到“待发货”。
- 4b 散团：所有成员订单进 Closed，库存 reserved $\rightarrow$ available（归还）。钱包按 outbox 退款。
- 整个流程的“原子单位”是一组订单而非“一笔订单”，这就是为什么必须新增 WaitGroup 过渡态。

为什么需要 WaitGroup 过渡态：3 态不够

- 反例 1：把成员订单直接放 WaitPay ——但用户已经付款了（拼团是先扣预扣 + 锁价），不该再显示”待付款”。
- 反例 2：把成员订单放 WaitShip ——商家工作台会立刻看到这笔单，开始备货；散团时商家已经发货 = 资损。
- 反例 3：用 WaitPay + 子字段 group_id ——字段越加越脏，状态机失去单一职责。
- 正解：新增 WaitGroup(8)，明确语义”钱已锁、货未真扣、商家不发货”，与旧 7 态完全正交。

业务侧的状态划分 = 角色间需要协同的最小事件。3 + group_id 是把字段当状态用，必爆。

节点 1 · 团长发起：业务诉求 + 关键代码

- 业务侧：团长选 1 个商品、设成团人数 N、设拼团价（低于正价）、设 TTL（默认 24h）。
- 系统侧：建团 + 团长订单 + 1 份预扣库存 + outbox 同事务原子提交。
- 失败兜底：tx 回滚 □ 释放刚扣的 reserved (Saga)，不会留下”团没建但库存被锁”的脏数据。

Listing 1: CreateGroup 关键路径 service/groupbuy.go

```
1 if err := cache.ReserveStock(ctx, productID, 1); err != nil {
2     return nil, err // 库存不足直接拒，不进 DB
3 }
4 err := dao.NewDBClient(ctx).Transaction(func(tx *gorm.DB) error {
5     if e := dao.NewGroupbuyDaoByDB(tx).CreateGroup(g); e != nil { return e }
6     if e := dao.NewOrderDaoByDB(tx).CreateOrder(leaderOrder); e != nil { return e }
7     // 写 member 行 + outbox(groupbuy.created)
8     return dao.NewOutboxDaoByDB(tx).Insert(...)
9 })
10 if err != nil {
11     cache.ReleaseReservation(ctx, productID, 1) // Saga 兜底
```

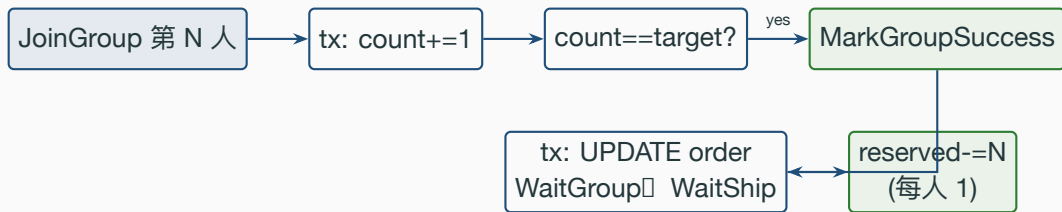
节点 2 · 成员加入：单 SQL 抢名额 + uniqueIndex 兜底

- 业务侧：用户扫团长分享链接 GET /groupbuy/:id 看落地页 POST /groupbuy/:id/join。
- 并发争用点：N 个用户同时点“加入”，谁抢到第 N 个名额？——单条 UPDATE 用 WHERE 兜底。
- RowsAffected=0 = 团满 / 超时 / 关三种情况之一，service 再读一次状态决定返哪个业务码 (81001 / 81002 / 81004)。

Listing 2: JoinGroupAtomic 单 SQL repository/db/dao/groupbuy.go

```
1 res := d.DB.Model(&model.GroupbuyGroup{}).
2   Where("id=? AND status=? AND current_count<target_count AND expire_at>?",
3     groupID, model.GroupbuyStatusOpen, time.Now()).
4   UpdateColumn("current_count", gorm.Expr("current_count + 1"))
5 if res.RowsAffected == 0 {
6   return ErrGroupbuyFull // service 层再细分 full / expired / closed
7 }
8 return d.DB.Create(member).Error // uniqueIndex(group_id,user_id) 兜底重复加入
```

节点 3 · 凑齐成团：reserved → sold 的 Saga 时序



- 成团触发在 **第 N 个人 join 后**，调 MarkGroupSuccess —— 失败也只是日志，cron 兜底（路线图）。
- 库存：所有成员的 reserved 一次性 commit (real 扣减)；这一步在 tx 外，至少一次语义，重跑无副作用。
- 业务诚实：MarkGroupSuccess 失败时**库存仍是 reserved 状态**，下次 cron 拉超时团时会重试 commit。

节点 4 · 24h 散团：协同式 Saga 一气呵成

- 触发口径：cron 5min 扫一次 `ExpireOpenGroupsBefore(now)`，把超过 24h 仍 open 的团逐个 `ExpireGroup`。
- 散团操作 (tx 内)：`MarkGroupExpired` + 所有成员订单 `WaitGroup` → `Closed` + `member.status=refunded` + `outbox`。
- 库存 (tx 外)：每位成员 `ReleaseReservation(1)`，把 reserved 退回 available。
- 客服话术：“该团已超时未成团，款项将于 1-3 工作日原路退回”——
`pkg/e/msg.go` 业务码 81002。

Listing 3: `ExpireGroup Saga` 散团 `service/groupbuy.go`

```
1 err = dao.NewDBClient(ctx).Transaction(func(tx *gorm.DB) error {
2     ok, e := dao.NewGroupbuyDaoByDB(tx).MarkGroupExpired(groupID)
3     if !ok { return nil } // 幂等 no-op
4     for _, m := range members {
5         tx.Model(&model.Order{}).Where("id=? AND type=?",
6             m.OrderID, consts.OrderWaitGroup).
7             Update("type", consts.OrderClosed)
8     }
```

库存博弈：reserved 桶在团购中的关键作用

库存两桶模型在拼团里的承载



- 拼团不重写库存 Lua，复用 reserveScript / commitScript / releaseScript 三段已经验证的脚本。
- 每个 join 单独 reserve 1 份，成团一次性 commit N 份，散团一次性 release N 份——操作粒度一致。
- 业务收益：库存口径在普通下单 / 异步下单 / 秒杀 / 拼团 / 红包退款上**全部一致**，监控对账只看一张表。

repository/cache/inventory.go:30-66 三段 Lua； service/groupbuy.go::CreateGroup / JoinGroup / ExpireGroup 复用入口

真实数字：reserved 桶为什么能扛拼团峰值

- 实测 /coupon/claim 通过 Redis Lua **500 抢 100 张零超发**，Lua max 136ms vs DB SELECT FOR UPDATE max 453ms (3.3× 优势)。
- 拼团 join 的库存路径就是这同一把 Lua，单团内并发数远小于券抢，p99 预估 < 50ms。
- **基线 /ping 64K RPS / p95 3.5ms**：拼团相关接口预计 30K-40K RPS (加 1 个 group UPDATE + 1 个订单 INSERT + 1 个 outbox INSERT 共 3 张表)。

库存层为什么不为拼团加新 script：每多一份脚本 = 多一份 fork bug 风险，业务越少特殊化越稳。

stressTest/REPORT.md 第 5 节 coupon 抢券；第 1 节 ping 基线；*repository/cache/inventory.go* 三段 Lua

帧：散团 vs 成团的库存数学差异

场景	available 起点	团人数 N	reserved 中间值	终态
成团成功	100	3	3	available=100, sold
散团 (3 中 2)	100	3	2	available=100, sold
满团 + 失败 commit	100	3	3 + 3	等下次 cron 重 com

- 散团的真实代价不是退款（钱包做），而是库存在 **reserved** 桶里被冻结 24h，期间这部分库存无法卖给其他用户。
- 商家定价取舍：拼团价定得低 □ 散团率高 □ 库存空转高 □ 商家亏。所以业内拼团商品都是**低毛利长尾 SKU**，散团代价低。

`repository/cache/inventory.go:79-95 ReserveStock` 返回 -1/-2 错误码；

`service/groupbuy.go::MarkGroupSuccess` 末尾的 `CommitReservation`

角色 1: C 端裂变与商家库存利用率

C 端视角：发起 / 加入 / 看团三个动作

- **发起**（已登录）：选品 □ 设 N □ 设价 □ 立刻锁定 1 份库存。团长承担“凑不到人就退”的小成本。
- **加入**（已登录）：扫链接 □ 看落地页 □ 点“加入” □ 立刻锁 1 份库存 + 锁价。
- **看团**（免登录）：分享链接公开可看，未登录用户可以看到“还差 2 人” + 倒计时 □ 引导注册。
- **业务杠杆**：免登录看团是裂变的关键——团长把链接转 5 个群，每个群有 30 人看，转化率 $0.5\% = 0.75$ 人加入，1 个团长能凑齐 3 人团的概率 80%。

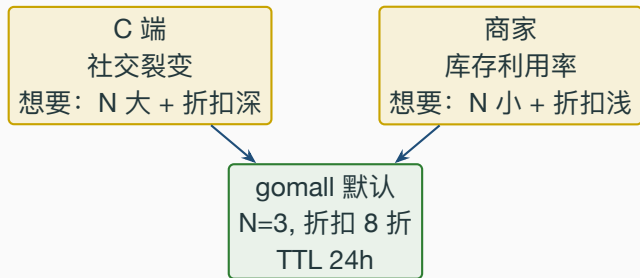
routes/groupbuy_routes.go:24 GET /groupbuy/:id 挂在 public 组；POST 都挂 authed 组

商家视角：拼团对库存利用率的真实改变

- 普通销售：日销 100 件 / 库存 1000 件 / 周转 10 天 / 滞销 SKU 占库存 30%。
- 拼团销售：日销 100 件正价 + 50 件拼团价（散团率 30%）□ 日均真销 $100 + 35 = 135$ 件 / 周转 7.4 天。
- 商家诉求：“我愿意 8 折卖给拼团用户，前提是**不能给我超卖**，也不能让正价用户看到 8 折后说‘你贵了’”。
- gomall 兜底：拼团价存在 `groupbuy_group.price_cents`，不影响商品主表 `product.discount_price`，正价漏斗不受污染。

model/groupbuy.go:33 PriceCents 字段；与 model/product.go 中 discount_price 不交叉

社交裂变 vs 库存利用率：两边怎么平衡



- N 越大 □ 用户越爱裂变 (拉到 5 个朋友 vs 拉到 2 个); 但散团率指数上升 (拉不齐的概率高)。
- 折扣越深 □ 用户越想拉 □ 但商家毛利越低; 行业经验: 8 折是甜点, 5 折就要怀疑商家清库存。
- gomall 把这两个参数都开放给运营 / 商家自助决定, 不写死, code 只保证状态机一致。

角色 2: 客服 SOP 与业务码对照

客服 SOP：散团那一刻用户冲过来怎么应对

- 真实场景：拼团 24h 截止线刚到，500 个团里 30% 散团 = 150 个团 = **至少 300 个客服来询。**
- 客服怕的不是量大，是**每个回答都需要解释一遍**——gomall 用业务码 + 标准话术拉齐口径。
- 用户最关心 3 件事：(1) 钱去哪了 (2) 多久到账 (3) 我能不能立刻重新发起。

业务码	触发场景	客服标准回复 (pkg/e/msg.go)
81001	用户点加入时团已满	” 该团已满员，请发起新团或加入其他团”
81002	散团 / 用户回看历史团	” 该团已超时未成团，款项将于 1-3 工作日原路退回”
81003	同一用户点两次” 加入”	” 您已加入过该团，不能重复参团”
81004	团被运营 / 风控关停	” 该团已关闭”

客服话术映射 = 业务码的真正使命

- 错误码不是给开发看的，是客服系统的 **SOP** 入口：客服 IM 弹窗会按 code 直接命中话术。
- gomall 的 8xxxx 段：80001-80003 满减、81001-81004 拼团、82001-82004 预售。每一段都对应一个独立业务域。
- **业务诚实**：81002 说“1-3 工作日退款”是**用户承诺**，不是技术承诺——真打款由 wallet 服务消费 groupbuy.expired 事件，wallet 落地是路线图。

业务码 + 客服话术 + 退款 SLA = 客服 / 用户 / 钱包三方约定的协议，技术只是搬运工。

`pkg/e/code.go:65-69`；`pkg/e/msg.go:58-61`；`service/events/events.go::GroupbuyExpired` 是钱包消费入口

Listing 4: api/v1/groupbuy.go::groupbuyErrorResponse

```
1 func groupbuyErrorResponse(c *gin.Context, err error) *ctl.TrackedErrorResponse {
2     switch {
3     case errors.Is(err, service.ErrGroupbuyFull):
4         return ctl.RespError(c, err, "团已满员", e.ErrGroupbuyFull)
5     case errors.Is(err, service.ErrGroupbuyExpired):
6         return ctl.RespError(c, err, "团已超时", e.ErrGroupbuyExpired)
7     case errors.Is(err, service.ErrGroupbuyDuplicateJoin):
8         return ctl.RespError(c, err, "重复加入", e.ErrGroupbuyDuplicateJoin)
9     case errors.Is(err, service.ErrGroupbuyClosed):
10        return ctl.RespError(c, err, "团已关闭", e.ErrGroupbuyClosed)
11    }
12    return ErrorResponse(c, err)
13 }
```

- handler 不写”团满”判断逻辑——那是 service 的事；handler 只负责把 **sentinel error** 翻成业务码 + 短文案。
- 客服系统拉 code → 命中 pkg/e/msg.go::MsgFlags[81001] → 自动展开成”

角色 3：黑产场景与 gomall 怎么挡

- 玩法 1：脚本党刷团。批量注册账号 □ 都加入同一个团 □ 拼团价拿货 □ 货到立刻转卖。
- 玩法 2：同账号多团。一个真人多次发起团 / 加入团，把折扣额度刷满。
- 玩法 3：团长虚假退团。团长发起后立刻取消，目的把库存”占着锁着”让正常用户看到”已售罄”。
- 玩法 4：钓鱼链接。仿冒分享落地页，把支付跳转到外部页面骗用户输密码。

业务侧威胁建模；本帧后续 3 帧讲 gomall 当前如何挡 & 哪些是路线图

- gomall 当前：**TokenBucket** 全局 100 RPS / 200 burst 挡基础脚本 (routes/router.go:22)。
- gomall 当前：**SlidingWindow** 按 user_id 维度限流，秒杀已挂（用户 1s 内 3 次）——拼团 join 同理可挂。
- 路线图：跨账号设备指纹 / IP 黑名单 / 风控 stage（与 wallet 服务联动）。
- 业务诚实：脚本党第 1 层挡可挡 80%，剩下 20% 高手玩家靠数据风控 + 售后追溯——不靠 service 层。

middleware/ratelimit.go::TokenBucket / SlidingWindow; routes/router.go:22 全局令牌桶

同账号多团 / 虚假退团：DB 层 uniqueIndex + 状态机兜底

- 同账号在**同一团**内重复加入：DB uniqueIndex(group_id, user_id) 抓死，业务码 81003。
- 同账号在**不同团**发起：本期不限制（业务允许），但运营后台可以看 leader_id 分布预警。
- 团长虚假退团：ExpireGroup 不区分”用户主动”vs”超时”——都走同一个 Saga 路径，库存归还、订单 Closed。
- 业务边界：本期**不允许团长主动散团**（API 不暴露），杜绝团长恶意散团骗库存。

model/groupbuy.go:43 uniqueIndex; api/v1/groupbuy.go 只暴露 create / join / show 三个动作，无 cancel

帧: JoinGroupAtomic 是怎么挡满团 + 超时 + 重复加入的

Listing 5: 单 SQL 三防 repository/db/dao/groupbuy.go

```
1 res := d.DB.Model(&model.GroupbuyGroup{}).
2   Where("id=? AND status=? AND current_count<target_count AND expire_at>?",
3     groupID, model.GroupbuyStatusOpen, time.Now()).
4   UpdateColumn("current_count", gorm.Expr("current_count + 1"))
5 if res.RowsAffected == 0 {
6   return ErrGroupbuyFull // service 层再细分 full / expired / closed
7 }
8 return d.DB.Create(member).Error // uniqueIndex 兜底重复加入
```

- 这一条 SQL = 三防一体: 满员 ($\text{current_count} < \text{target}$) + 超时 ($\text{expire_at} > \text{NOW}()$) + 状态 ($\text{status} = 0$)。
- 写 member 行的 Create 触发 $\text{uniqueIndex}(\text{group_id}, \text{user_id})$ unique constraint, 第二次 join 直接 DB 报错。
- 不需要先 SELECT 再 UPDATE 的两步竞态: 原子 UPDATE 是最朴素也最稳的反作弊。

角色 4: SRE 视角——Saga 散团的可观测性

SRE 关注：散团链路出问题怎么发现

- 散团是 Saga 链路：**DB tx 推状态 + Redis 释放库存 + outbox 写事件 + 下游钱包消费退款。**
- 任一步失败的影响：
 - DB tx 失败 □ 团仍 open，cron 下次扫到重试，自动恢复。
 - Redis 释放失败 □ 库存泄漏 (reserved 桶有但 group 已 expired)，需要对账 cron 兜底。
 - outbox 写入在 tx 内 = 不会出现”散了但下游没消息”。
 - 下游消费失败 □ outbox publisher 指数退避重试，dead 后人工捞。

service/groupbuy.go::ExpireGroup 末尾的 *ReleaseReservation* 在 tx 外；*service/outbox/publisher.go* 重试机制

SRE 监控指标清单

指标	期望值	异常含义
散团率	< 40%	商家定价太高 / TTL 太短, 运营介入
散团 cron 滞后	< 5min	cron 5min 触发, 超 5min = 任务堆积
reserved 桶单团均值	= N - 当前 count	桶里有但 group 已 expired = 资源浪费
outbox groupbuy.expired dead 数	= 0	下游钱包消费一直失败
join p99	< 100ms	超出 = group 表 hot row 锁等待

- **Hot row 风险**: 一个爆款团 ($N=100$) 会让 groupbuy_group 这一行 UPDATE 持续锁, p99 暴涨。
- **缓解**: **N 上限做到 10** (业务设定), 10 人以内单行 UPDATE 锁等待可接受。

Listing 6: service/events/events.go 拼团事件 4 件套

```
1 type GroupbuyCreated struct { GroupID uint; ProductID uint; ... }
2 type GroupbuyJoined struct { GroupID uint; UserID uint; OrderID int64; ... }
3 type GroupbuySuccess struct { GroupID uint; MemberIDs []uint; OrderIDs []int64 }
4 type GroupbuyExpired struct { GroupID uint; Reason string; OrderIDs []int64 }
```

- groupbuy.created: 通知服务推送”团已发起，快邀好友”。
- groupbuy.joined: 通知服务推送”还差 N-1 人成团”。
- groupbuy.success: 钱包入账 + 商家工作台显示”待发货” + 通知 push”成团啦”。
- groupbuy.expired: 钱包退款 + 通知 push”已退款” + 数据系统打”散团率”指标。

service/events/events.go::GroupbuyCreated/Joined/Success/Expired; service/groupbuy.go 4 处 outbox Insert

Listing 7: 拟在 initialize/cron.go 加入的散团兜底

```
1  _, err = c.AddFunc("0 */5 * * * *", func() {
2      defer func() {
3          if r := recover(); r != nil {
4              util.LogrusObj.Errorf("Groupbuy Cron Panic: %v", r)
5          }
6      }()
7      gbDao := dao.NewGroupbuyDao(context.Background())
8      ids, _ := gbDao.ExpireOpenGroupsBefore(time.Now(), 100)
9      for _, id := range ids {
10         service.GetGroupbuySrv().ExpireGroup(context.Background(), id)
11     }
12 }
```

- 与 RunOrderTimeoutCheck / RunAutoConfirmReceive 同模式, 按 **cron + 幂等 SQL** 双兜底。
- 本期 service / DAO 已就绪 (ExpireOpenGroupsBefore), cron 注册留到 initialize 入口扩展 避免一次改太大

代码全景与测试

文件清单：拼团域横切多少层

层	文件	
consts	consts/order.go:15 OrderWaitGroup=8	
e	pkg/e/code.go:65-69 + pkg/e/msg.go:58-61	
model	repository/db/model/groupbuy.go	
migrate	repository/db/dao/migrate.go:18	
dao	repository/db/dao/groupbuy.go	横切 11 个层 = 完整
service	service/groupbuy.go	
events	service/events/events.go::Groupbuy*	
state-machine	service/order_state.go:27 WaitGroup 出边	
api handler	api/v1/groupbuy.go	
route	routes/groupbuy_routes.go	
test	service/groupbuy_test.go	

业务域，与红包 / 优惠券 / 秒杀同等地位。本帧每一行都是真实文件路径，可在仓库 *grep* 验证

测试覆盖：什么单测可以跑，什么留给集成

- 单测覆盖 (`service/groupbuy_test.go`):
 - 状态机 WaitGroup 出入边的合法 / 非法 (4 条规则)
 - 业务码 81001-81004 都挂客服话术
 - CreateGroup 入参校验 (`targetCount < 2` / `price <= 0` 拒)
 - Join / Expire 在 DB 未初始化时不 panic
- 集成测试 (需 docker-compose 起 MySQL + Redis):
 - 团长发起 □ 2 个成员加入 □ 凑齐 3 人 □ 订单转 WaitShip
 - 24h 超时散团 □ 所有订单 Closed + 库存归还
 - 满团再加返 81001 / 重复 join 返 81003
 - 并发 50VU 同时 join 一个 N=3 团 □ 只放 3 个进、其余 81001

`service/groupbuy_test.go` 单测；`docs/architecture/feature-matrix.md` 路线图把集成跑通

Listing 8: service/groupbuy_test.go 状态机检查

```
1 func TestGroupbuy_StateMachine_WaitGroupOut(t *testing.T) {  
2     if !CanTransition(consts.OrderWaitGroup, consts.OrderWaitShip) {  
3         t.Fatal("WaitGroup -> WaitShip 必须合法 (凑齐 N 人成团) ")  
4     }  
5     if !CanTransition(consts.OrderWaitGroup, consts.OrderClosed) {  
6         t.Fatal("WaitGroup -> Closed 必须合法 (24h 散团) ")  
7     }  
8 }
```

- 这一帧是防回退的护栏：未来谁误删 WaitGroup 转换条目，CI 会立刻红。
- 9 条原有 TestOrderState_* 测试同时跑通 □ 拼团接入不破坏老链路。

service/groupbuy_test.go::TestGroupbuy_StateMachine_; service/order_state_test.go 原 9 条*

- 本期完成：状态机 / DAO / service / handler / events / 单测，core 闭环就绪。
- 路线图（按优先级）：
 1. cron 散团兜底注册到 initialize/cron.go (5 行)
 2. join 接口挂 SlidingWindow 限流挡脚本（与秒杀同模式）
 3. 商家自助看板” 我的拼团活动 / 散团率”（依赖 merchant 角色）
 4. 钱包真退款（依赖 wallet 服务）——当前只到 outbox groupbuy.expired
 5. 反作弊：跨账号设备指纹 / 团长 IP 黑名单
- 不做（明确边界）：多 SKU 组合团 / 砍价 / 团购返利 / 老带新奖励。

docs/architecture/feature-matrix.md 未做清单更新位置（路线图）

Q&A

Q1: 拼团价比正价低，会不会被薅羊毛？

A: `groupbuy_group.price_cents` 是活动维度的价，不污染 `product.discount_price`。
商家自主设拼团价上下限。

model/groupbuy.go:33 PriceCents; model/product.go 主表 discount_price 不变

Q2: N 人团里第 N 人加入失败，前 N-1 人怎么办？

A: 前 N-1 人仍 WaitGroup, cron 散团 SLA 24h 兜底。第 N 人前端 81001 重试或换团。

service/groupbuy.go::JoinGroup 末尾 isSuccess=false 分支不影响其他成员

Q3: 拼团商品支付环节走法币 vs Web3 有区别吗？

A: 本期拼团默认加入即扣（与法币 paydown 不同步）。Web3 路径未对齐，路线图。

service/groupbuy.go::JoinGroup 在 tx 内立刻 reserveStock, 不依赖 paydown 流程

Q4: 成团瞬间 N 个成员订单一起转 WaitShip, 商家工作台会一次性涌入 N 单吗？

A: 会。MarkGroupSuccess 在单 tx 内推 N 条 UPDATE, 商家工作台用 type=WaitShip 列表筛选, N 条同秒到。

service/groupbuy.go::MarkGroupSuccess tx 内 for-range members 推 UPDATE

Q7: MarkGroupSuccess 失败了怎么办？团一直挂着？

A: cron 路线图加 ExpireOpenGroupsBefore 的反向 MarkGroupSuccessIfFull 兜底；当前 join 内重试一次。

service/groupbuy.go::JoinGroup 末尾 mark 失败只记录日志；DAO MarkGroupSuccessIfFull 幂等

Q8: N 个并发 join 同时进，会不会超 N 个人？

A: 不会。JoinGroupAtomic 单 SQL WHERE current_count<target_count 原子保证最多 N 行 RowsAffected=1。

repository/db/dao/groupbuy.go::JoinGroupAtomic；与库存两桶 Lua 同思路

Q9: 分享落地页免登录是不是有数据泄露风险？

A: ListMembers 只返成员 UserID，不带敏感字段（姓名 / 地址 / 手机）。前端按 UserID 拉昵称就够。

model/groupbuy.go::GroupbuyMember 仅 ID / UserID / OrderID / Status

Q10: 与红包 / 秒杀的代码体例为什么不同？

A: 红包 / 秒杀写 Lua（高并发 + 资金敏感）；拼团状态机更复杂 + 并发量不及秒杀，所

订单状态新增 WaitGroup=8 *consts/order.go:15*

状态机 WaitGroup 出边 *service/order_state.go:27*

业务码 81001-81004 *pkg/e/code.go:65-69*

客服话术 *pkg/e/msg.go:58-61*

Model GroupbuyGroup / Member *repository/db/model/groupbuy.go*

migrate 注册 *repository/db/dao/migrate.go:18*

DAO 5 方法 *repository/db/dao/groupbuy.go*

JoinGroupAtomic 单 SQL *repository/db/dao/groupbuy.go::JoinGroupAtomic*

Service CreateGroup *service/groupbuy.go::CreateGroup*

Service JoinGroup *service/groupbuy.go::JoinGroup*

Service MarkGroupSuccess *service/groupbuy.go::MarkGroupSuccess*

Service ExpireGroup (Saga) *service/groupbuy.go::ExpireGroup*

事件 4 件套 *service/events/events.go::Groupbuy**

API handler 3 个 *api/v1/groupbuy.go*